

第 1 報：DSPy_Shizuoka_V3

多様な最適化問題への DSPy + GEPA 強化学習の適用と
参照解を上回るアルゴリズム生成の試み

DSPy_Shizuoka_V3 プロジェクト

2026 年 8 月 4 日

概要

本報告は、静岡向け最適化問題集 V3 (30 問, 7 カテゴリ) を対象とし、DSPy パイプラインに GEPA (Genetic-Pareto Evolutionary Prompt Adaptation) を組み合わせて、参照解 (OR-Tools ベースライン) を上回る `solve(instance)` 関数の自動生成を目指した試行の記録である。V2 (DSPy_Shizuoka_V2) との主要な差分は、(a) 問題タイプが 4 種から 7 種へ拡張され、参照解 (`reference_solution`) の構造も問題ごとに大きく異なる点、(b) 評価指標を「実行可能性 + 参照解ギャップ」から「参照値を上回った件数 (`beat_reference`) の最大化」に振り切った点である。バグ修正後のベースラインを起点に、4 つのフェーズ (A/B/C/D) を段階的に適用し、訓練セット (24 問) での `beat_reference` 件数を $4 \rightarrow 7 \rightarrow 5 \rightarrow 9 \rightarrow 8$ と改善した。特にフェーズ C の「汎用参照解ガイド型メトリック」により平均スコア $0.520 \rightarrow 0.901$, テスト集合の `beat_reference` $0 \rightarrow 1$ (`prob_030` で -80%) を達成した。一方でフェーズ D では GEPA が実際にプロンプトを進化させ (Program 0 $\rightarrow$ Program 4), HARD RULES とキー名例, OR-Tools 特有の構文注意点を含む構造化された指示文が獲得された。

目次

1	本報の位置づけ	1
2	V3 システム全体像	2
2.1	データ	2
2.2	パイプライン	2
2.3	評価メトリック	2
3	V2 との差分吸収	2
4	改良フェーズ A/B/C/D	3
4.1	ベースライン (バグ修正のみ)	3
4.2	フェーズ A: スコア自己計算 + 参照解要約	3
4.3	フェーズ B: <code>gen_error</code> 修正 + Signature 強化 + 生 JSON	3
4.4	フェーズ C: 汎用参照解ガイド型スコアラ (最良)	3
4.5	フェーズ D: GEPA チューニング (4 項目)	4
4.6	フェーズ横断比較	5
5	GEPA の効果とプロンプト進化	5
5.1	フェーズ C まで: 進化ゼロの理由	5
5.2	フェーズ D で獲得された進化プロンプト	5

5.3	トレードオフ	6
6	フェーズ E：100 問データ・40/20 分割・demonstrations・目的検出修正	6
6.1	改良 1：目的フィールド自動検出の修正	6
6.2	改良 2：GEPA demonstrations の導入	7
6.3	改良 3：Signature 薄型化と exec_error 抑制の両立	7
6.4	結果	7
6.5	進化後プロンプトの成長	7
7	参照フリー・スコアリングの検証	8
7.1	動機	8
7.2	実装：3 つのリーク経路の遮断	8
7.3	比較の指標	8
7.4	結果	8
7.5	知見	9
8	考察	9
8.1	最も効いた改良	9
8.2	GEPA が働くための条件	9
8.3	テスト集合での上回り件数	9
9	結論と今後の課題	10

1 本報の位置づけ

姉妹プロジェクトである DSPy_Shizuoka (第 1 報), DSPy_Shizuoka.V2 は, TSP / CVRP / OP / JSSP の 4 タイプに対して DSPy + MIPROv2 を適用し, 参照解に対する相対ギャップの最小化を目的としていた。本 V3 では以下の点で状況が異なる。

1. **問題の多様化**。スケジューリング (動的計画法・整数計画・混合整数計画・グラフ最適化・確率最適化) と配送・輸送 (混合整数計画・確率最適化) の合計 7 カテゴリを含む 30 問を扱う。各問題の reference_solution のフィールド名 (roster, gate_assignment, lineup, timetable, trains, machine_assignment など) は問題ごとに異なり, V2 のような固定スキーマは適用できない。
2. **評価目的の転換**。「参照値と一致すれば十分」ではなく, **参照値を上回る (beat reference) 件数**を可能な限り増やすことが本 V3 の主目的である。
3. **学習アルゴリズムの変更**。MIPROv2 から GEPA [1] に切り替えた。GEPA は反射的フィードバック (*reflective feedback*) を利用してプロンプトを世代進化させる点で MIPROv2 とは異なる。

以下, 第 2 節で本 V3 の全体構成, 第 3 節で V2 との差分吸収, 第 4 節で改良フェーズ A/B/C/D, 第 5 節で GEPA の効果とプロンプト進化, 第 8 節で考察, 第 9 節で結論と今後の課題を述べる。

2 V3 システム全体像

2.1 データ

data/prob_001.json ~ prob_030.json の 30 問を用いる。各ファイルには name, core_type, requirement, instance, reference_solution, reference_value が含まれる。訓練 / テスト分割は先頭

24 問を訓練 (prob_001~prob_024), 残り 6 問をテスト (prob_025~prob_030) とした*¹。

2.2 パイプライン

パイプラインは V2 とほぼ同じであるが, `parse_instance` は動的 LLM 呼び出しではなく静的な `parse_code` を用いる形に変更した (フェーズ B 参照)。

```
requirement 自然言語 () + core_type + reference_solution 要約
-> AlgorithmGenerator.forward
  -> generate.predict (dspy.ChainOfThought)
    -> algorithm_code (Python source)
      -> safe_exec -> solve(instance) -> solution
        -> metrics_v3.evaluate_algorithm_v3 -> score
```

2.3 評価メトリック

評価関数 `evaluate_algorithm_v3` は以下の `status` を返す。スコアは V2 と同じく参照値との相対比を用いるが, 最終的な最適化目的は `beat_reference` 件数である。

表 1 `status` と代表的なスコア

status	代表スコア	意味
beat_reference	1.5 ~ 2.5	参照値を厳密に上回った
exact_match	1.5	参照値と一致
new_best	1.0 ~	過去最良を更新
similar	~ 1.0	参照値と同等
partial_feasible	~ 0.5	一部制約のみ満足
worse	≤ 0	参照値より悪い
infeasible	0	制約違反
invalid_solution	0	構造不一致 (キー名不整合など)
exec_error	0	実行時例外
gen_error	0	LLM 生成失敗

3 V2 との差分吸収

V2 から V3 への移行に際しては以下の変更を最小限で行った。

1. `data_loader`: V2 の `examples.jsonl` 一括読み込みから, `prob_NNN.json` 個別ファイルの読み込みに変更。
2. `core_type` 対応: 7 カテゴリの日本語 `core_type` をそのまま `InputField` として渡す。
3. `reference_solution`: 問題ごとに構造が違うため, V2 の固定 `INSTANCE_SCHEMAS` 方式は放棄。代わりに `requirement_builder.summarize_reference_solution` が実行時にスキーマを推論し, `requirement` テキスト末尾に添付する (フェーズ A)。
4. スコア関数: V2 の TSP/CVRP/OP/JSSP 個別スコアラは V3 では不十分。フェーズ A で 5 種のスコアラを自己計算型に書き直し, フェーズ C で汎用スコアラ `generic_ref_guided_cost` を導入。

*¹ 当初 100 問を想定していたが, 実データは 30 問だったため 80/20 分割ではなく 24/6 分割に修正した。

4 改良フェーズ A/B/C/D

本節では、バグ修正後のベースラインから始め、順に 4 つの改良フェーズを適用した記録を示す。各フェーズの結果は表 2 に一括して示す。

4.1 ベースライン (バグ修正のみ)

初期実装には次の 2 つのバグがあった。

Bug#1: `cost=0` の偽解。 空の を返すコードが「コスト 0」として `new_best` 判定され、実質的な信頼できない `beat_reference` を大量に生んでいた。

Bug#2: f-string 内部の変数参照ミス。 デバッグログ出力で `{score}` が `{status}` とミスタイプされ、一部の `status` が上書きされていた。

未利用: `reference_solution`。 LLM の入力プロンプトに参照解の構造情報を渡していなかった。

これらを修正した状態を「ベースライン」とし、訓練セットで 4 件の `beat_reference` と平均スコア 0.520 を得た。ただし 14 件は依然として `cost=0` の偽解が `first_valid` として計上されていた。

4.2 フェーズ A: スコア自己計算 + 参照解要約

変更点。(i) 5 種のスコアラ (`utils/scorer.py`) を、LLM が返す `cost` フィールドではなく解の生構造から自己計算する形に書き直す。空解に対しては `None` を返し、`first_valid` に格上げされないようにした。(ii) `requirement_builder.summarize_reference_solution` で参照解のキー名・型・サイズを要約し、`requirement` 末尾に追加。

結果。訓練 `beat_reference` = 7, 平均 0.471。偽解 `new_best` は消えたが、`invalid_solution` が 13 件と目立ち始めた。これは 7 タイプの多様な参照解構造に、5 種の固定スコアラが十分対応できていないことを示す。

4.3 フェーズ B: `gen_error` 修正 + Signature 強化 + 生 JSON

変更点。(i) 元実装では `modules.forward` 内で `parse_instance` を `DSPy` の動的呼び出しにしていたが、これが `gen_error` (kwarg 衝突) を頻発させていた。静的な `parse_code` に置き換えて解消。(ii) `GenerateOptimizationAlgorithm` Signature を強化し、返却キー名の厳密一致・タイムアウト・フォールバックの必要性を明記。(iii) `instance` の生 JSON を最大 6000 文字で `requirement` に埋め込み、LLM がスキーマを自力で把握できるようにした。

結果。`gen_error` は消滅。しかしフェーズ A の反動で Signature の指示が強くなりすぎ、平均 0.372 とスコアはむしろ悪化、`beat_reference` も 5 件に減少。一方で テスト集合 の平均は $-0.117 \rightarrow 0.082$ と正の方向に転じた。

4.4 フェーズ C: 汎用参照解ガイド型スコアラ (最良)

変更点。`metrics.v3.py` に `generic_ref_guided_cost` を追加。参照解の構造から自動的に「割当フィールド」と「目的値フィールド」を検出する。

```
_MIN_KEYWORDS = {'cost', 'time', 'makespan', 'tardiness',
                  'distance', 'delay', 'loss', 'risk', ...}
_MAX_KEYWORDS = {'profit', 'revenue', 'score', 'rating',
                  'utility', 'reward', 'satisfaction', ...}
```

```
def generic_ref_guided_cost(sol, ref):
    # 1. Detect assignment field: 最大の非スカラー・非"note"フィールド
    assign_key = _find_largest_container(ref)
    # 2. Detect objective field: 数値スカラーで_MIN/_MAX_KEYWORDS を含むキー
    obj_key, is_min = _find_objective(ref)
    # 3. Extract sol_val from solution using the same keys
    sol_val = sol.get(obj_key)
    ref_val = ref.get(obj_key)
    # 4. Return cost so that lower-is-better (min case: return sol_val
    # directly; max case: return ref_val**2 / sol_val to invert)
    ...
```

この機構により、lineup / roster / gate.assignment などどの構造でも同一関数で処理でき、最小化・最大化の混在にも自動対応できる。

結果。 フェーズ全体を通じて **最良**。訓練の `beat_reference = 9`, `new_best = 5`, 平均スコア 0.901 (フェーズ B の 0.372 から +0.53)。テストでも `beat_reference = 1` (`prob_030` で -80%), 平均 0.593 と大幅改善。

4.5 フェーズ D: GEPA チューニング (4 項目)

フェーズ C までは Signature 側とスコアラ側の改良のみで、GEPA の内部 (プロンプト進化) はほとんど働いていなかった。実際 `compiled_program.metadata` を確認すると、85 反復のすべてで「Program 0 (ベース Signature)」が最良として保持されていた。

原因を「ベース Signature が強すぎる + 予算が小さい + valset が小さい」と整理し、以下 4 項目を同時適用した。

1. **ベース Signature の薄型化。** `GenerateOptimizationAlgorithm` を約 4300 文字 → 約 1276 文字に縮小。「型 (Signature) は最小限、中身 (指示) は GEPA が発見」との方針。
2. **予算拡大。** `breadth` を 3 → 6, `depth` を 5 → 8 (`max_evals` 15 → 48)。
3. **valset 拡大。** `n_val` を //5 → //3 (4 例 → 8 例)。
4. **反射的フィードバックの強化。** `gepa_feedback_v3.py` に `summarize_ref_solution` を追加し、`invalid_solution` / `worse` / `infeasible` / `partial_feasible` の各ケースで参照解構造ヒント (`ref_hint`) をフィードバック本文に注入。

結果。 今回初めて GEPA がプロンプトを実際に進化させ、Program 4 (進化後) が Program 0 (ベース) を凌駕した。訓練の `beat_reference` は 3 件だが `exact_match` を 5 件含めると 8 件が「参照値以上」を達成。平均は 0.602 (フェーズ C の 0.901 より劣化。理由は `exec_error` が 1 → 6 件に増加したため)。テストは `beat_reference = 1` (`prob_028` で -30%)。

4.6 フェーズ横断比較

表 2 各フェーズの訓練 (24 問) およびテスト (6 問) 結果

Phase	Train mean	Beat_ref	Exact_match	Test mean	Test beat_ref	GEPA 進化
Baseline (bug fix)	0.520	4	—	—	—	なし
A (scorer 自己計算)	0.471	7	—	-0.117	0	なし
B (Signature 強化)	0.372	5	—	0.082	0	なし
C (汎用スコアラ)	0.901	9	3	0.593	1 (<code>prob_030</code> -80%)	なし
D (GEPA チューニング)	0.602	3	5	0.399	1 (<code>prob_028</code> -30%)	あり

5 GEPA の効果とプロンプト進化

5.1 フェーズ C まで: 進化ゼロの理由

フェーズ C までの detailed.results を精査したところ、GEPA は反復ごとにプロンプト候補を提案しているが、評価集合上のスコアが常にベース (Program 0) と同点かそれ以下となり、選択されなかった。原因は次の 3 点である。

- ベース Signature がすでに指示を詰め込みすぎており、改良の余地が小さい。
- max_evals=15 という予算が「1 世代 3 提案 × 5 深さ」相当と狭く、Pareto フロントを十分に広げられない。
- n_val = train / 5 = 4 と極端に小さく、候補間の差が測定ノイズに埋もれる。

5.2 フェーズ D で獲得された進化プロンプト

フェーズ D で GEPA が最終的に選択した Program 4 は、約 6.4K 文字の高度に構造化された指示文である。以下はその抜粋 (原文英語)。

```
### HARD RULES (Never Violate)
1. Function Signature: Define exactly one top-level function: def solve(instance).
2. Return Value: You MUST return the solution object. NEVER use print().
3. Structure Match: The returned object MUST strictly match the "Reference
   Solution Structure" or "Return Format" specified in the input.
   - Critical: Pay close attention to the exact key names in the reference
     structure.
   - Example: If the reference shows {'lineup': {...},
     'total_expected_rating': 147.55}, your return value MUST use the key
     'lineup', NOT 'assignment' or 'schedule'. Mismatched keys result in a
     score of 0.
4. Runtime Budget: Maximum 30 seconds. Set OR-Tools
   solver.parameters.max_time_in_seconds = 20 (or less).
5. Robustness: Wrap the main algorithm in a try...except block.
6. Syntax Correctness: In OR-Tools CP-SAT, do NOT write
   model.Add(condition) for item in items. Instead use a loop.

### Algorithm Strategy Guidelines
1. Problem Analysis:
   - Small Instances (< 50 vars/constraints): Prefer OR-Tools CP-SAT or PuLP.
   - Medium/Large Instances: Use Greedy + Local Search (2-opt, swap).
   - Graph Problems (Critical Path, Shortest Path): Use Topological Sort,
     Dijkstra, or Bellman-Ford.
2. Data Parsing:
   - Handle missing keys with .get(key, default).
   - Map categorical data (shift names, genres) to integer indices.
3. Constraint Handling:
   - Capacity/Resource Limits, Uniqueness, Min/Max counts.
   - For "no more than 2 consecutive night shifts,"
     use logical implications or auxiliary variables in CP-SAT.
4. Fallback Heuristic Design:
   - Implement a deterministic greedy fallback that always returns a
     valid non-empty solution.

### Output Format
### reasoning
<step-by-step reasoning>

### algorithm_code
```

注目すべきは次の 3 点である。

1. 参照解キー名の具体例 ('lineup' vs 'assignment') が直接文字列で埋め込まれた。これはフェーズ D の ref_hint フィードバックが実問題での失敗を学習した結果と考えられる。
2. OR-Tools 構文特有の落とし穴 (model.Add(cond) for item in items が構文エラーである旨) が明記されている。これは exec_error のフィードバックから学習された。
3. 問題規模に応じたアルゴリズム選択 (小: CP-SAT / 中: Greedy+LS / グラフ: Dijkstra 等) がガイドラインとして体系化されている。

5.3 トレードオフ

進化後のプロンプトは指示が具体的である一方、**冗長すぎて** LLM が指示の一部を無視・混同するケースが増え、実測では exec_error がフェーズ C の 1 件から 6 件に増加した。このため **平均スコアはフェーズ C より低下している**。参照値を上回る **件数** という主目的ではフェーズ C と D はほぼ同等 (訓練で 9 vs 8) だが、**安定運用** の観点ではフェーズ C の設定が推奨される。

6 フェーズ E：100 問データ・40/20 分割・demonstrations・目的検出修正

フェーズ D までは V3 の 30 問 (24 訓練 / 6 テスト) を用いたが、テスト集合が小さく運要素が大きいという課題があった。フェーズ E では新規に整備した **100 問データセット** (OptimizationDataCreator/data, 27 コアタイプ) を導入し、**層化抽出による 40 訓練 / 20 テスト分割** (load_and_split_stratified, seed=42, disjoint, 各分割に 20 コアタイプ) へ移行した。同時に、次の 4 つの改良を実施した。

6.1 改良 1：目的フィールド自動検出の修正

generic_ref_guided_cost は参照解の構造から「どの数値フィールドが目的値か」を推定するが、フェーズ D までは単純なキーワード照合であり、複数の数値指標や時系列を含む複合構造 (trains, makespan と peak_resource_usage が併存する等) で誤検出していた (例：問題 20 が peak_resource_usage でなく makespan を、問題 65 が min_rolls でなく total_waste を目的と誤認)。

これを解消するため、スコアリング方式の select_objective_field() を新設した。各候補フィールドに対し以下の加点・減点を行い最大スコアを選ぶ。

- min_ / max_ 接頭辞：+5
- 目的文 (objective) のトークン一致 (日英対応)：+4
- 集約接頭辞 total_/peak_/expected_/best_/optimal_：+2
- 解にキーが存在：+1
- 値がコンテナ要素数と一致 (構造的カウント)：-4
- カウン트의な名前：-1

方向 (最小化/最大化) は min_/max_ 接頭辞、次いで目的文の「最大/最小」、最後にキーワードで決定する。目的文は data_loader が各 Example に付与し (objective フィールド)、メトリックまで流す。複数目的候補を持つ 10 問で検証し **10/10 正解**を確認した。

6.2 改良 2：GEPA demonstrations の導入

dspy.GEPA は指示 (instructions) のみを進化させ、few-shot 例は保持する。この性質を利用し、既知良コード (問題 001 の `exact_match`, 問題 021 の `beat_reference`) を `program.generate.predict.demos` へ手動付与した。これにより、`exact_match` 型と `beat_reference` 型の双方の実例がプロンプトに常在し、GEPA の指示進化と両立する。

6.3 改良 3：Signature 薄型化と `exec_error` 抑制の両立

フェーズ D で増加した OR-Tools 構文起因の `exec_error` を抑えるため、`GenerateOptimizationAlgorithm` に **HARD RULE #6 (SYNTAX SAFETY)** を追加した。括弧の対応、および `model.Add(expr) for x in items` 形式の末尾ジェネレータ禁止 (実ループを使う) を明記する。これは詳細指示を GEPA に委ねつつ、構文安全性のみを最小限のハードルールで担保する二段構成である。

6.4 結果

100 問・40/20 分割での評価結果を表 3 に示す。フェーズ D (30 問・24/6) とはデータが異なるため厳密な同一比較ではないが、テストの参照超え率・平均スコアともに大幅に向上した。

表 3 フェーズ D と フェーズ E の比較

指標	フェーズ D (24/6)		フェーズ E (40/20)	
	訓練	テスト	訓練	テスト
平均スコア	0.602	0.399	0.859	0.800
参照超え件数	8/24	1/6	20/40	7/20
参照超え率	33%	17%	50%	35%
有効解 (score>0)	14/24	2/6	29/40	12/20

テスト 20 問の内訳は `exact_match` 6, `new_best` 2, `beat_reference` 1, `similar` 3, `worse` 7, `exec_error` 1, `parse_error` 1 であり、`exec_error` は **1 件のみに**抑えられた (フェーズ D は 6 件)。HARD RULE #6 が有効に機能したと言える。

6.5 進化後プロンプトの成長

フェーズ E で進化した `generate` の instructions は約 **17.3 KB** に達し、GEPA の反射ループが失敗事例から**問題タイプ別の解法知識**を体系的に蓄積した。特筆すべきは末尾の「**Critical Learning from Previous Failures**」節で、実際の失敗から学んだ参照解キー名の具体例が多数埋め込まれた点である。

- **Bottleneck Assignment** : 目的キーは `bottleneck_time` ではなく `min_bottleneck`, 出力は `['min_bottleneck', 'assignment', 'note']`。解法もソート済みコスト上の二分探索＋二部マッチングと明記。
- **Parallel Machine Scheduling** : `machine_assignment` は Machine ID でなく **Job ID** をキーとする。
- **Staff Rostering** : `roster` は Time slot でなく **Staff ID** をキーとし、未割当スタッフも空リストで含める。

- **ライブラリ耐性**：pulp/scipy の import 失敗を前提に純 Python フォールバック必須。

これらは `ref_hint` と `exec_error` フィードバックが実問題での失敗を学習した直接の成果であり、V3 の「参照解が問題ごとに全く異なる構造を持つ」という本質的困難に対して有効に作用した。

7 参照フリー・スコアリングの検証

7.1 動機

フェーズ E までの報酬関数は、参照値 (`reference_value`) を用いた `exact_match / beat_reference` ボーナスを中核としていた。しかし実運用では**参照値が存在しない新規問題**へ適用する場面が想定される。そこで、参照値を報酬・フィードバックから完全に排除し、強化学習的に「自ら得た解の改善」のみを報酬とする**参照フリー・モード** (`--no-reference`) を実装し、参照ありモードと参照超え件数を比較した。

7.2 実装：3 つのリーク経路の遮断

「参照フリー」を厳密に成立させるには、参照値が学習に混入する 3 経路すべてを断つ必要がある。

1. **報酬**：`compute_v3_score(use_reference=False)` で `exact_match / beat_reference` ボーナスと参照ベースの `suspicious` 検出を無効化。代わりに初回実行可能コストを**セルフベースライン**として正規化アンカーに用いる。
2. **best_known 初期化**：参照値からの seeding を停止し、ディスク上の参照シード済み `best_known.jsonl` を読み込まず、空の `best_known_noref.jsonl` から開始。
3. **フィードバック文**：`gepa_feedback_v3` で参照解の数値目標行を隠蔽 (`hide_values=True`)。出力スキーマ (トップレベルのフィールド名) のみ提示し、これは「問題仕様」であって「解答」ではないと整理した。

なおコスト測定は両モードで同一とするため、目的フィールドの特定には参照解の**構造** (どのキーが目的値か) を引き続き用いる (値そのものは使わない)。これにより、比較は「参照値を報酬に使うか否か」のみを操作した対照実験となる。

7.3 比較の指標

両モードを公平に比較するため、スコア式に依存しない**事後** (*post-hoc*) **指標**を定義した。実行可能かつ実コストを持つ結果について、 $\text{cost} \leq \text{ref}$ を「参照以上 (at-or-better)」, $\text{cost} < \text{ref}$ を「厳密超え (strict)」と数える (`exec_error`, `infeasible` 等は除外)。この指標はスコアリング・モードから独立で、同一テスト分割 (`seed=42`) 上で `per-problem` に一致する。

7.4 結果

表 4 に、参照あり (フェーズ E) と参照フリー (フェーズ F) の head-to-head を示す。

7.5 知見

- テストの参照超え件数は完全に一致 (7/20, 厳密 1/20)。参照値を報酬から取り除いても参照超え件数は 100% 維持された。訓練でもわずかな低下 (22→20, 9→8) にとどまる。
- 有効解率はむしろ向上 (テスト 12/20 → 19/20)。制約充足+自己改善という報酬が「まず正しく動く解」を強く促した結果と解釈できる。

表 4 参照あり vs 参照フリー (同一の事後指標, 100 問 40/20, seed=42)

指標	参照あり (E)		参照フリー (F)	
	訓練	テスト	訓練	テスト
参照以上 ($\text{cost} \leq \text{ref}$)	22/40	7/20	20/40	7/20
厳密超え ($\text{cost} < \text{ref}$)	9/40	1/20	8/40	1/20
有効解	29/40	12/20	37/40	19/20
exec_error (テスト)		1		1

- 平均スコアはモード間で報酬スケールが異なるため直接比較しない。公平な軸は上記の事後指標のみである。

この結果は、GEPA が外部ターゲットとしての参照値なしでも、feasibility ゲートと best_known / セルフベースラインに対する改善という RL 的シグナルだけで参照超えを引き出せることを示す。すなわち本パイプラインは参照値の無い新規問題にも適用可能である。

8 考察

8.1 最も効いた改良

コード変更で見ると、フェーズ C の generic_ref_guided_cost が群を抜いて有効であった (beat_ref 4 → 9)。これは V3 の reference_solution が問題ごとに全く異なる構造を持つという本質的な特性に対して直接効いた変更である。逆に言えば、V2 では成立していた「タイプごとに固定スコアラ」戦略が V3 では通用しないことを実測で確認した。

8.2 GEPA が働くための条件

フェーズ A/B/C では GEPA プロンプト進化が発生しなかった。プロンプト進化を起こすためには少なくとも「(a) ベース Signature を薄く」「(b) 予算 max_evals を数十以上」「(c) valset を訓練集合の 1/3 以上」の 3 条件が必要と示唆される。フェーズ D の設定はこの条件を満たしたため、初めて Program 4 が Program 0 を Pareto フロントで凌駕した。

8.3 テスト集合での上回り件数

最終的にテスト集合 (6 問) で参照値を上回れたのはフェーズ C・D それぞれ 1 件のみである。特に prob_025, prob_026, prob_029 はどのフェーズでも worse または infeasible のままだった。これらは参照値がかなり厳しく設定されており、LLM のゼロショット生成では届かない可能性が高い。Few-shot 例示 (V2 の MIPROv2 相当) の再導入が今後の候補となる。

9 結論と今後の課題

本 V3 プロジェクトでは、多様な参照解構造を持つ 30 問に対し DSPy + GEPA で参照値上回り件数の最大化を試みた。

- バグ修正 → A → B → C → D の 4 フェーズを通じて、訓練 beat_reference を 4 → 9 (フェーズ C), テスト beat_reference を 0 → 1 に改善。

- フェーズ C の `generic_ref_guided_cost` が最大の効果を発揮。
- フェーズ D で初めて GEPA がプロンプトを進化させ、HARD RULES と OR-Tools 構文警告を含む構造化された指示文を獲得した。
- フェーズ D は指示文が冗長化した結果 `exec_error` が増加し、平均 スコアではフェーズ C に劣る。実運用には **フェーズ C の設定** を推奨する。
- **参照フリー・モード**を追加し、参照値を報酬・フィードバックから完全に排除しても、テストの参照超え件数が 7/20 (厳密 1/20) と参照ありモードに完全一致することを実証した (§7)。参照値の無い新規問題への適用可能性を示す。

今後の課題として次を挙げる。

1. Few-shot 例示の再導入 (MIPROv2 併用または GEPA の `demonstrations` 機能利用)。
2. `generic_ref_guided_cost` で誤検出されるケース (`trains` など時系列を含む複合構造) への対応。
3. Signature 薄型化と `exec_error` 抑制の両立 (例: 短い HARD RULES のみベースに残し、詳細指示は GEPA に任せる二段構成)。
4. テスト集合の拡大 (V3 は 6 問しかなく、運要素が大きい)。

参考文献

- [1] L. Agarwal, et al. *GEPA: Genetic-Pareto Prompt Optimization as an Alternative to Reinforcement Learning for LLM Agents*. Preprint, 2024.